Forecasting Overnight Minimum Temperature to Decide When to Run Cold Protection

Time-Series Forecasting Project — Advance Pattern Recognition

[Group members: Ikram Kakar, Amit Negi]

Submitted: 14, November 2026

Code and data repository:
https://github.com/ikramkakar/time-series-forecasting-melbourne

The problem. Each evening a small plant nursery in Melbourne must decide whether to run overnight cold protection (a heater or frost covers over cold-sensitive seedlings), using tomorrow's forecast minimum temperature. We call a night below 6°C a cold night. This threshold is our assumption: we assume cold-sensitive seedlings begin to suffer damage near this level, and we fixed the value after counting how many nights fall below several candidates, because the test period has no night below 0°C and only 2 below 2°C. In the 548-night test period (July 1989 to December 1990), 49 nights (8.9%) were cold. Let C be the cost of acting for one night (heating fuel or the labour of covering the seedlings) and L the loss from one missed cold night (damaged or lost seedlings, replanting, delayed sales).

The recommendation. If a missed cold night costs more than roughly 7 to 10 times a night of acting (L/C above about 7 to 10), use the LSTM prediction-interval rule: act whenever the model's 10% lower-quantile forecast falls below 6°C. If L/C is below about 7, use the free habit "act tonight if last night's minimum was below 6°C"; the model is not worth deploying there. Point forecasts alone are not enough: the best point models (RMSE about 2.2°C) catch only 20 to 37% of cold nights.

The expected saving. Costs below are for the 548 test nights, in units of C, with the rule chosen on validation data. At $L/C = 10$ the interval rule costs 263 against 490 for doing nothing, a **46% saving**, and 22% less than the persistence habit (339). At $L/C = 15$ and 20 the savings against doing nothing are 62% and 70% (43% and 53% against the habit). With illustrative figures $C = 20$ and $L = 200$ currency units, the 548 nights cost 5,260 instead of 9,800 (4,540 saved) or 6,780 for the habit (1,520 saved). These money figures are assumptions, not measured costs.

The risk. The saving depends on L/C. At $L/C = 10$, a block bootstrap (14-day blocks, 2,000 resamples) gives a 5th to 95th percentile saving of +23% to +60% against doing nothing, but only −1% to +38% against the habit. If the true L/C is about 5, the interval rule loses to the habit by a median 28%. The rule flags 233 of 548 nights (43%) and 187 of those flags are false alarms; it misses 3 of the 49 cold nights. Choosing the rule on validation instead of on test cost at most about 10 to 11% more than the best rule. The 95% interval itself is well calibrated (coverage 95.4%, average width 9.6°C). These results come from one test period containing 49 clustered cold nights, so treat the percentages as indicative.

Test period, units of C	$L/C = 5$	$L/C = 10$	$L/C = 20$
Do nothing	245	490	980
Persistence habit	194	339	629
LSTM point forecast rule	184	339	649
LSTM interval rule (10% quantile)	248	263	293

Contents

1 Introduction and decision context **3**

2 Data and exploratory analysis **3**

2.1 Data and cleaning (Task 1) 3

2.2 Train, validation and test split (Task 3) 3

2.3 Decomposition (Task 2) 4

2.4 Autocorrelation (Task 2) 4

2.5 Stationarity (Tasks 6 and 7) 5

3 Classical (linear) models **5**

3.1 Evaluation protocol 5

3.2 Choosing p and q (Tasks 4 and 5) 6

3.3 ARIMA (Task 8) 6

3.4 Seasonal model (Task 9) 7

4 Neural network models (Tasks 10 and 11) **7**

4.1 Test-set comparison of all models 8

5 From forecasts to action (Task 12) **10**

6 Probabilistic forecast (Task 13) **11**

7 Risk and reliability (Task 14) **12**

8 Management assessment **13**

9 Conclusion **14**

AI disclaimer **15**

Appendix: reproducibility **15**

1 Introduction and decision context

This project forecasts the daily minimum temperature one day ahead and asks a practical question: when does a forecast change what an operator should do? The decision we study is whether to run overnight cold protection for [site type] on a given evening. Acting on a night costs C (energy, staff time or materials). Failing to act on a night that turns out cold costs L (damaged stock or equipment, a call-out, lost work the next morning). Because we do not have a measured L and C for a real site, we express every result as a function of the ratio L/C and show for which ratios a forecasting model is worth using.

We define a cold night as a minimum below 6°C. This threshold was chosen after looking at how many nights in the data fall below 0, 2, 4 and 6°C (Section 5), so it is a stated assumption and not a result. All fourteen tasks in the course brief are covered: data analysis (Tasks 1 to 3), classical models (4 to 9), neural networks (10 and 11), and the decision, probabilistic forecast and risk analysis (12 to 14).

All code, the data file and the result tables are in the repository at https://github.com/ikramkakar/time-series-forecasting-melbourne.

2 Data and exploratory analysis

2.1 Data and cleaning (Task 1)

We use daily minimum temperatures in Melbourne, Australia, from 1 January 1981 to 31 December 1990: 3,652 days (see reference [9]). The raw file needed cleaning: some temperature values carried a "?" prefix, and a footer line of metadata sat at the end of the file. We removed the prefix, dropped the footer, converted dates and temperatures to proper types, enforced a continuous daily index and filled any gaps by linear interpolation. The series minimum is 0.0°C, and only 28 nights in the whole series are below 2°C.

2.2 Train, validation and test split (Task 3)

The series is split in time order, with no shuffling, and the same split is used by every model. Every model makes one-step-ahead predictions: to predict day t, it sees the true observations up to day $t - 1$.

Table 1: Chronological split.

Set	Days	From	To
Training (70%)	2,556	1 Jan 1981	31 Dec 1987
Validation (15%)	548	1 Jan 1988	1 Jul 1989
Test (15%)	548	2 Jul 1989	31 Dec 1990

The test window covers one and a half years, so it includes a full seasonal cycle. Test data were not used to choose any model setting; orders, harmonics, network settings and probability levels were selected on validation data.

2.3 Decomposition (Task 2)

An additive decomposition with a yearly period (365 days) gives the variance shares in Table 2. The yearly cycle dominates: it explains 57.8% of the variance with a peak-to-trough swing of 11.98°C. The trend is almost flat (1.0% of the variance, ranging only from 10.46 to 12.20°C), which agrees with the unit root results in Section 2.5. The three shares add to 97.8%, not 100%, because the trend and residual are undefined at the edges and the components are not exactly uncorrelated.

Table 2: Variance shares from the additive decomposition.

Component	Share of variance	Note
Seasonal	57.8%	Peak-to-trough 11.98°C
Trend	1.0%	Range 10.46 to 12.20°C
Residual	39.0%	Mean 0.001, std 2.543°C

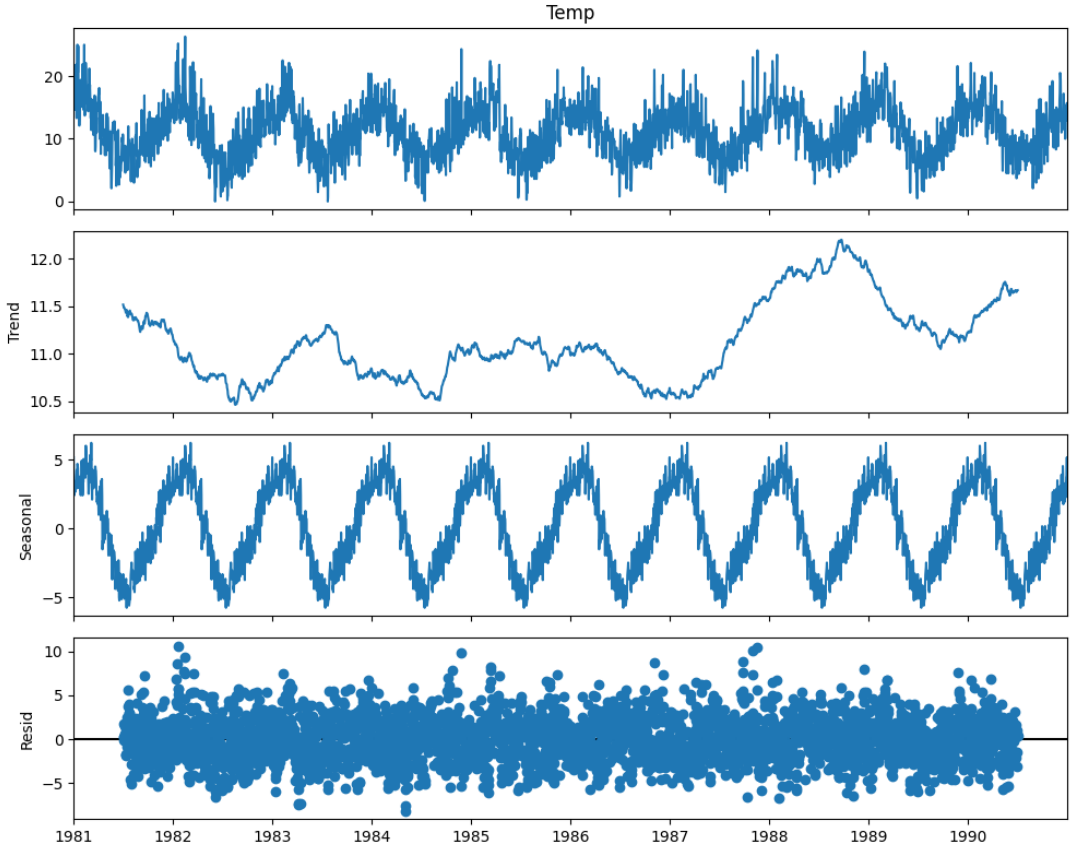


Figure 1: Additive decomposition into trend, seasonal and residual components.

2.4 Autocorrelation (Task 2)

The raw autocorrelation decays slowly (0.765 at lag 1, 0.616 at lag 2, 0.570 at lag 3) even though the series is stationary; the slow decay comes from the seasonal cycle and not from a unit root. At lag 180 the autocorrelation is -0.444 and at lag 365 it is $+0.457$: half a year apart gives opposite seasons, a full year gives the same season. There is no weekly pattern: the value at lag 7 (0.556) simply continues the decay from lag 3, so we did not use a period-7 seasonal model. The residual

after removing trend and season is not white noise: its autocorrelation is 0.496 at lag 1 and 0.177 at lag 2, then drops to about 0.07. A warm or cold anomaly therefore tends to persist for a few days, and this is the part a short autoregressive model can exploit.

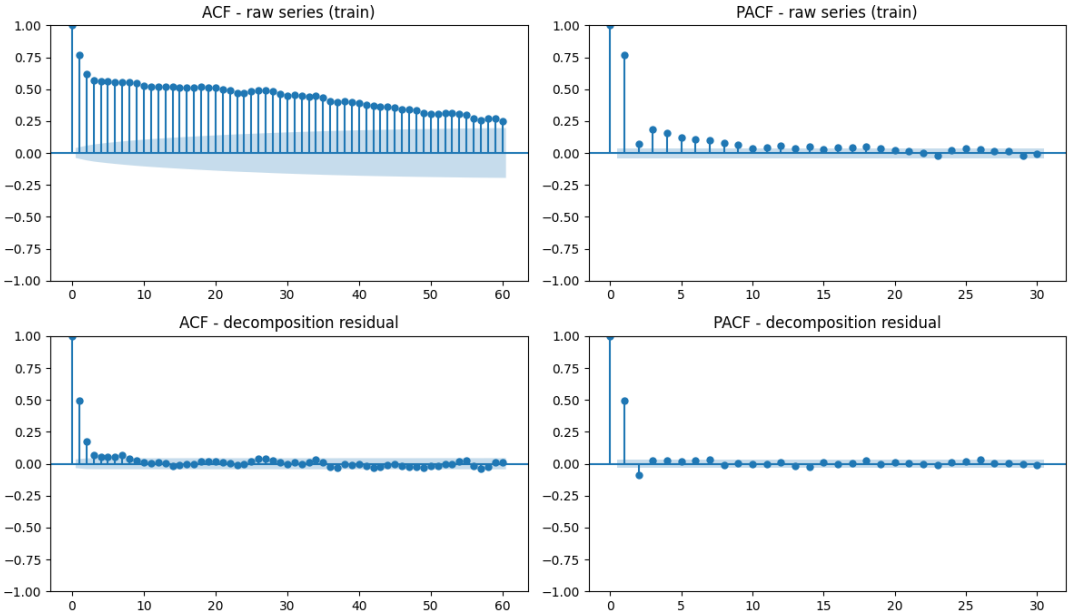


Figure 2: ACF and PACF of the training series and of the decomposition residual.

2.5 Stationarity (Tasks 6 and 7)

On the training set, the augmented Dickey–Fuller test rejects a unit root (statistic -3.928, $p = 0.0018$) and the KPSS test does not reject stationarity (statistic 0.116; the p-value of 0.10 is the limit of the lookup table, so it should be read as $p \geq 0.10$). The two tests, which have opposite null hypotheses, agree. We therefore use $d = 0$. The yearly cycle is deterministic seasonality around a fixed pattern, not non-stationarity. The first-differenced, seasonally differenced and combined series also pass both tests (ADF statistics -18.4, -18.7 and -15.8), but differencing a stationary series is unnecessary and removes the persistence that short AR models rely on, so we did not difference the data.

Table 3: Unit root tests on the training set.

Series	ADF stat	ADF p	KPSS stat	KPSS p
Raw	-3.928	0.0018	0.116	≥ 0.10
First difference ($d = 1$)	-18.445	0.0000	0.296	≥ 0.10
Seasonal difference (365)	-18.660	0.0000	0.199	≥ 0.10
Seasonal + first	-15.830	0.0000	0.063	≥ 0.10

3 Classical (linear) models

3.1 Evaluation protocol

Each model is fitted on the training set only. Its parameters are then frozen and applied to the full series so that each prediction uses the true past values, which gives genuine one-step-ahead

forecasts on validation and test. Reporting multi-step forecasts would be misleading here: an AR or ARMA forecast many days ahead collapses toward the mean. We report RMSE, MAE, MAPE, sMAPE and R^2. MAPE is reported because the brief asks for it, but it is unreliable for this series: on nights near 0°C the denominator is tiny and the error explodes, so we add sMAPE. A persistence forecast (tomorrow equals today) is the reference every model must beat.

3.2 Choosing p and q (Tasks 4 and 5)

The theory says q is read from where the ACF cuts off and p from where the PACF cuts off. Our PACF has no clean cut-off. With 2,556 training days the 95% band is only ± 0.039, and lags 1 to 9, 11 to 14 and 16 to 18 are all outside it (lag 1: 0.765, lag 2: 0.073, lag 3: 0.188, lag 4: 0.153). Because the PACF does not give a clear answer, we used the validation set. Table 4 shows pure AR(p) and pure MA(q) fits.

Table 4: Pure AR and MA models: AIC (training fit) and RMSE (validation, one-step-ahead).

Order	AR AIC	AR val RMSE	MA AIC	MA val RMSE
1	12194.6	2.468	13049.8	3.069
2	12183.2	2.456	12638.3	2.804
3	12091.3	2.415	12490.2	2.660
4	12032.7	2.405	12399.8	2.608
5	11997.4	2.374	12328.3	2.575
6	11969.7	2.361	12285.6	2.544
7	11944.7	2.357	12243.9	2.507
8	11930.4	2.357	12214.0	2.487

What decides p. AIC keeps falling up to order 8, but validation RMSE stops improving at $p = 7$ (2.357, the same at $p = 8$). We take the smaller order, $p = 7$. This is not a weekly effect, since the ACF shows no weekly cycle; it is just where the improvement ends. **What decides q.** Validation RMSE for the MA model falls from 3.069 at $q = 1$ to 2.487 at $q = 8$ and is still falling. There is no ACF cut-off because the autocorrelation decays slowly, so a long MA is needed to imitate what a short AR does. We fix $q = 8$, the largest order we tested, and note that this is a limit of our search and not an optimum.

3.3 ARIMA (Task 8)

Because $d = 0$, the ARIMA family reduces to ARMA(p, q). We fitted all orders with p and q from 0 to 3 on the training set. The top models are statistically tied: the best validation RMSE is 2.323 for $(2, 2)$, and the top seven are within 0.015°C, with AIC within about 2 points of each other. We selected **ARIMA(2,0,2)** for its lowest validation RMSE and small parameter count. Dropping the second MA term, ARIMA(2,0,1), has almost the same validation RMSE (2.333) but an AIC about 35 points worse, so we kept it.

Table 5: Best ARMA(p, q) orders on validation.

(p, q)	AIC	Validation RMSE
(2, 2)	11864.3	2.323
(3, 1)	11862.0	2.324
(3, 3)	11866.0	2.325
(3, 2)	11864.4	2.325
(1, 3)	11862.2	2.326
(2, 1)	11899.3	2.333

3.4 Seasonal model (Task 9)

A seasonal ARIMA with period 365 is too heavy to fit, and a period of 7 is wrong for this data. Instead we model ARMA(2,2) errors with yearly sine and cosine terms (Fourier terms) as regressors. We tried 1 to 4 harmonics and selected the number on validation data.

Table 6: Number of Fourier harmonics K, ARIMA(2,0,2) errors.

K	AIC	Validation RMSE
1	11764.9	2.284
2	11765.5	2.278
3	11768.7	2.277
4	11770.6	2.264

Adding the seasonal terms lowers the in-sample AIC by about 100 points (11864 to about 11765), so the yearly cycle is statistically real. On new data the gain is smaller, because yesterday's temperature already carries much of the seasonal information. AIC prefers $K = 1$ and validation RMSE prefers $K = 4$; the differences are small, and following the brief we chose $K = 4$ on validation.

4 Neural network models (Tasks 10 and 11)

The neural models use the same split. The input is scaled with a scaler fitted on the training set only. Each window of the previous *lag* days predicts the next day, and windows are built so that validation and test targets are exactly the same calendar days as in the classical models (we checked this in code). We trained SimpleRNN and LSTM networks with the Adam optimiser [7], mean squared error loss and early stopping on validation loss. For each network type we searched 12 random configurations from lag in $\{3, 7, 14, 30\}$, units in $\{16, 32, 64\}$, layers in $\{1, 2\}$, learning rate in $\{0.001, 0.003, 0.01\}$ and batch size in $\{16, 32, 64\}$. This is a small random search and not an exhaustive grid, and each configuration was run with one random seed.

Table 7: Top five RNN configurations on validation.

Lag	Units	Layers	LR	Batch	Epochs	Val RMSE
30	16	1	0.001	32	60	2.307
14	16	1	0.001	32	47	2.325
30	16	2	0.003	64	25	2.333
7	32	1	0.003	16	31	2.366
7	64	1	0.001	32	14	2.367

Table 8: Top five LSTM configurations on validation.

Lag	Units	Layers	LR	Batch	Epochs	Val RMSE
30	16	2	0.003	64	42	2.288
30	16	1	0.001	32	100	2.292
14	16	2	0.010	16	24	2.305
30	64	1	0.003	32	38	2.311
14	16	1	0.001	32	84	2.312

Observations. The lag mattered most: all of the top LSTM configurations use a lag of 30 or 14, and lag 3 was worst. Small networks were enough: the top three RNN configurations all use 16 units, so extra capacity added no accuracy. The top two configurations of each type differ by less than the noise one random seed could cause, so we do not claim that the chosen settings are uniquely best. Lag 30 was the largest value in our search space, so a longer window could help further. The best LSTM used lag 30, 16 units, 2 layers, learning rate 0.003 and batch size 64; the best RNN used lag 30, 16 units, 1 layer, learning rate 0.001 and batch size 32. One LSTM run used all 100 epochs, so the epoch cap may have cut it short.

4.1 Test-set comparison of all models

Table 9: One-step-ahead results on the test set (validation RMSE for reference).

Model	RMSE	MAE	MAPE %	sMAPE %	R^2	vs persist.	Val RMSE
ARIMA(2,0,2) + Fourier, $K = 4$	2.184	1.720	19.49	17.26	0.686	−15.1%	2.264
LSTM (tuned)	2.208	1.721	18.70	17.50	0.679	−14.2%	2.288
ARIMA(2,0,2)	2.233	1.739	19.44	17.50	0.672	−13.2%	2.323
AR(7)	2.261	1.759	19.84	17.64	0.664	−12.1%	2.357
RNN (tuned)	2.265	1.769	19.60	17.92	0.663	−11.9%	2.307
AR(1)	2.428	1.928	21.87	19.23	0.612	−5.6%	2.468
MA(8)	2.431	1.927	22.18	19.16	0.611	−5.5%	2.487
Persistence (yesterday)	2.572	2.036	22.35	21.03	0.565	–	n/a

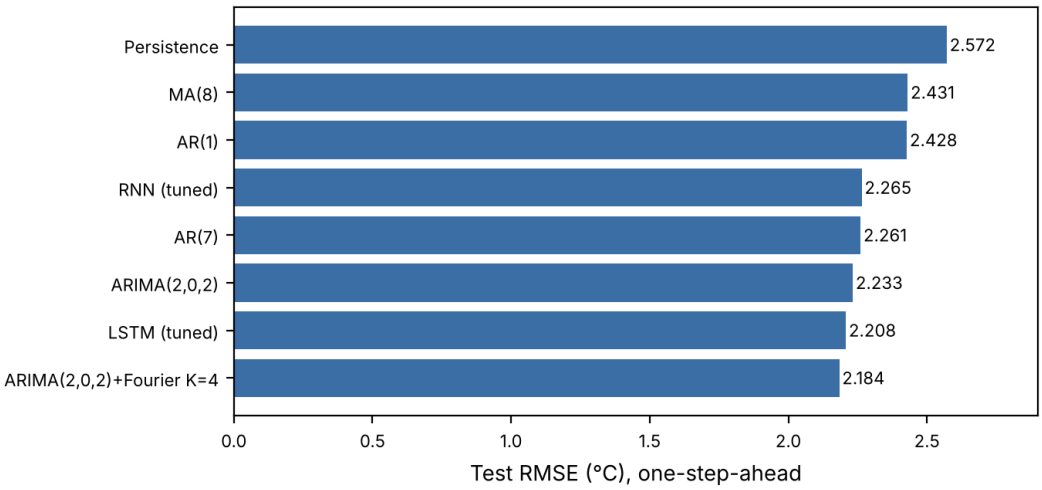


Figure 3: Test RMSE of every model (one-step-ahead, July 1989 to December 1990).

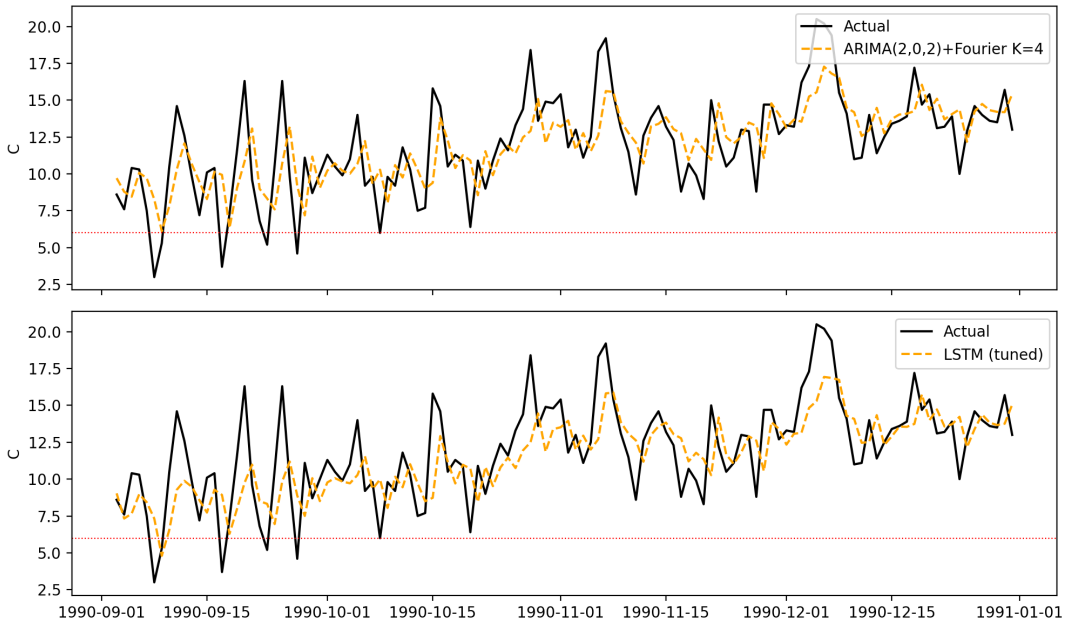


Figure 4: Predicted versus actual minimum temperature on the test set for the two best models.

Observations. (1) Every fitted model beats persistence, and the best (ARIMA with Fourier terms) reduces test RMSE by 15.1%. (2) The tuned LSTM is within 1.1% of the best linear model (2.208 against 2.184), which is too small a gap to call a real difference with one seed per configuration. Non-linearity therefore adds little for one-step-ahead daily temperature: yesterday's value and the season do most of the work. (3) The seasonal ARIMA is told the day of the year, while the networks must infer it from at most 30 days of history. (4) MA(8) with eight parameters is no better than AR(1) with one, so a pure moving-average model suits this persistent series poorly. (5) MAPE hardly separates the models (19% to 22%) because of nights near 0°C; RMSE and sMAPE are more informative. (6) The test period is slightly easier than validation for every model, so comparisons should be made within one split and not across them.

5 From forecasts to action (Task 12)

A low RMSE does not by itself say whether a forecast improves a decision. We therefore turn each forecast into a yes/no action. A rule acts on a night when its forecast is below 6°C. The total cost of a rule over the test period is

$$\text{cost} = (\text{nights acted}) \times C + (\text{cold nights missed}) \times L.$$

We compare seven rules: do nothing; a persistence habit (act if last night's minimum was below 6°C); the point forecast of ARIMA+Fourier; the point forecast of the LSTM; and three LSTM lower-quantile rules at the 2.5%, 10% and 20% levels.

The reason for the quantile rules is a standard result: a rule that acts when the τ-quantile forecast falls below the threshold is cost-optimal when $\tau = C/L$. At $L/C = 10$ the right level would be 10%; at $L/C = 40$ it would be 2.5%. The point forecasts aim at the middle of the distribution, which is the wrong place to look for cold extremes.

The test set has 0 nights below 0°C, 2 below 2°C, 11 below 4°C and 49 below 6°C. Counts at the lower thresholds are too small for a cost analysis, which is why we use 6°C.

Table 10: How each rule handled the cold nights (threshold 6°C, 548 test nights, 49 cold).

Rule	Nights acted	Cold nights caught	Missed	False alarms
Point: ARIMA+Fourier	11	10	39	1
Point: LSTM	29	18	31	11
Persistence habit	49	20	29	29
Interval: $\tau = 20\%$	143	37	12	106
Interval: $\tau = 10\%$	233	46	3	187
Interval: $\tau = 2.5\%$	293	49	0	244

Observations. The point forecasts catch only 10 and 18 of the 49 cold nights (20% and 37%). Both models pull forecasts toward the typical value for the season, so extremes are under-predicted. The model with the lowest RMSE is therefore the weakest decision tool at most cost ratios. At the other extreme the 2.5% rule catches every cold night but acts on 53% of all nights. Only 0.9% of test nights fell below the 2.5% quantile forecast, 6.4% below the 10% one and 15.1% below the 20% one, so the lower tails are conservative: the 10% model behaves like a 6% quantile.

Table 11: Total cost on the test set (units of C).

Rule	$L/C = 5$	$L/C = 10$	$L/C = 20$
Do nothing	245	490	980
Persistence habit	194	339	629
Point: ARIMA+Fourier	206	401	791
Point: LSTM	184	339	649
Interval: $\tau = 2.5\%$	293	293	293
Interval: $\tau = 10\%$	248	263	293
Interval: $\tau = 20\%$	203	263	383

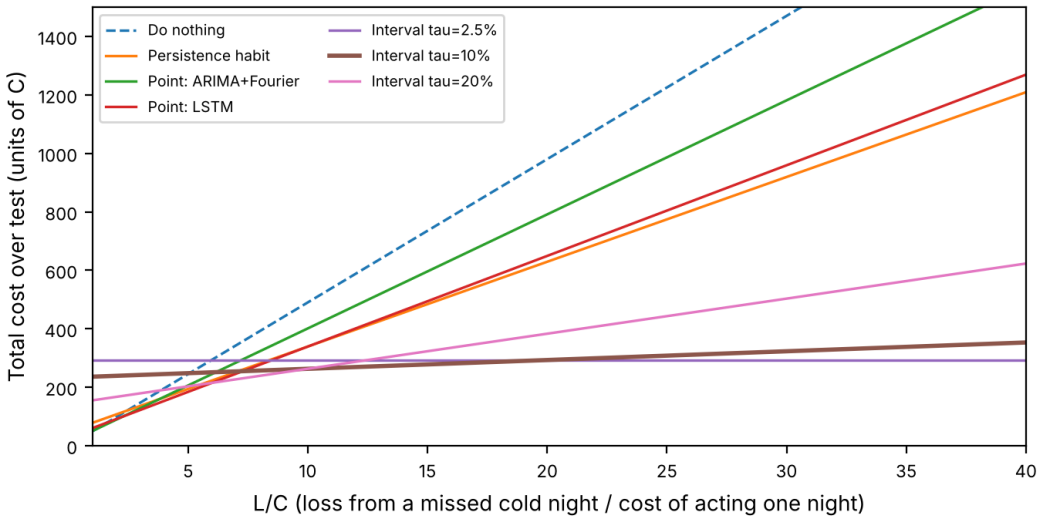


Figure 5: Total cost of each rule as the loss ratio L/C rises (threshold 6°C, test period).

Break-even ratios. Acting on the persistence habit beats doing nothing above $L/C = 2.45$. The 10% interval rule beats the persistence habit above $L/C = 7.08$ and the LSTM point rule above 7.29. The LSTM point rule beats the persistence habit only below $L/C = 10$, and by a small margin. This gives three regimes: below about 2.5 do not act on forecasts; between about 2.5 and 7 the simple habit is nearly as good as the model; above about 7 use the interval rule with τ close to C/L.

Choosing the rule without peeking at the test set. To check that these conclusions do not come from looking at the test cost, we chose the cheapest rule on validation at each L/C and then measured its cost on test (Table 12). The rule chosen on validation was never worse than the persistence habit on test, and its extra cost relative to the best rule on test (the regret) was 0 at four of the seven ratios and at most 30 units (about 10 to 11% of the best cost). Validation contained only 30 cold nights, so the exact regime boundaries are approximate.

Table 12: Rule chosen on validation, cost measured on test (units of C).

L/C	Rule chosen on validation	Test cost	Do nothing	Persistence	Saving vs nothing	Saving vs persist.	Regret
3	Point: LSTM	122	147	136	17.0%	10.3%	0
5	Persistence	194	245	194	20.8%	0.0%	10
7	Persistence	252	343	252	26.5%	0.0%	25
10	Interval $\tau = 10\%$	263	490	339	46.3%	22.4%	0
15	Interval $\tau = 10\%$	278	735	484	62.2%	42.6%	0
20	Interval $\tau = 10\%$	293	980	629	70.1%	53.4%	0
30	Interval $\tau = 10\%$	323	1470	919	78.0%	64.9%	30

6 Probabilistic forecast (Task 13)

We trained two LSTM networks, one for the 2.5% quantile and one for the 97.5% quantile, by minimising the pinball loss [6]. Together they form a 95% prediction interval. Both use the best LSTM settings from Section 4 (lag 30, 16 units, 2 layers, learning rate 0.003, batch size 64), scaled inputs, and early stopping on validation loss. We evaluate with the prediction interval coverage probability (PICP, the share of actual values inside the interval) and the mean prediction interval width (MPIW).

Table 13: Interval quality for the 95% interval.

Set	PICP	MPIW (°C)	Below lower	Above upper	Crossings
Validation	95.62%	9.59	1.5%	2.9%	0
Test	95.44%	9.56	0.9%	3.6%	0

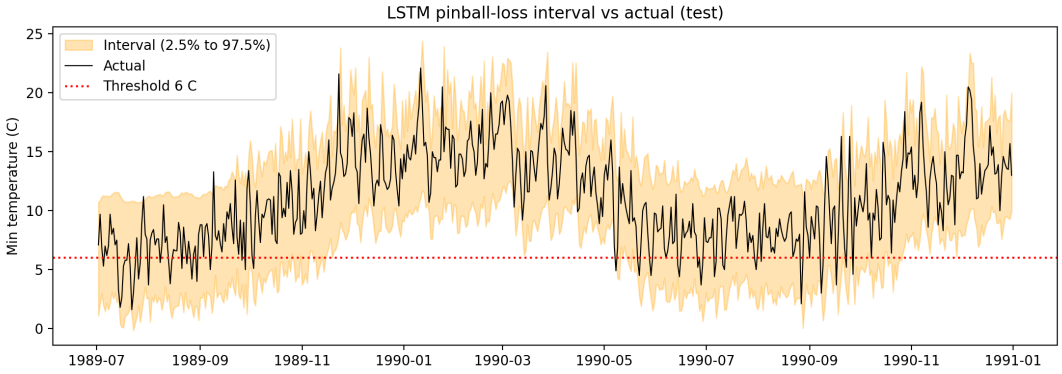


Figure 6: 95% LSTM prediction interval against actual minimum temperature, test period, with the 6°C threshold.

Observations. Coverage is close to the 95% target on both sets, so the interval is well calibrated overall. With 548 days and strongly correlated neighbouring days, the sampling error on a coverage figure is about one percentage point or more, so we describe it as close to nominal and not as exact. The two tails are not symmetric: only 0.9% of test nights fall below the lower bound (the target is 2.5%) while 3.6% fall above the upper bound (the target is 2.5%), and the validation set shows the same pattern. The lower bound is conservative, which matters for a cold-night decision. The average width of 9.56°C is about 10% wider than a Gaussian interval built from the point-forecast RMSE ($2 \times 1.96 \times 2.208 \approx 8.7$°C). The interval is wide, about the size of the yearly swing of 12°C, because daily minimum temperature is inherently noisy. The two models never crossed, so the lower bound was always below the upper bound. Validation data were used for early stopping, so the validation figures are slightly optimistic; the test figures are the ones to quote.

7 Risk and reliability (Task 14)

The test costs above come from one period with 49 cold nights, and these nights occur in runs rather than independently. To quantify how much luck is involved, we resampled the test period 2,000 times in blocks of 14 consecutive days (a moving block bootstrap, see reference [8]) and recomputed each rule's saving. The block length of 14 days was our choice.

Table 14: Saving of the 10% interval rule and of the LSTM point rule, block bootstrap (median, 5th to 95th percentile).

L/C	Rule	Saving vs doing nothing	Saving vs persistence habit
5	Interval $\tau = 10\%$	-3% (-44 to $+24$)	-28% (-66 to -4)
5	LSTM point	$+24\%$ ($+11$ to $+35$)	$+5\%$ (-2 to $+12$)
10	Interval $\tau = 10\%$	$+46\%$ ($+23$ to $+60$)	$+22\%$ (-1 to $+38$)
10	LSTM point	$+30\%$ ($+17$ to $+41$)	$+0.3\%$ (-9 to $+8$)
20	Interval $\tau = 10\%$	$+70\%$ ($+56$ to $+78$)	$+54\%$ ($+37$ to $+65$)
20	LSTM point	$+33\%$ ($+19$ to $+44$)	-3% (-14 to $+6$)

How the interval turns into reliability. The interval adds value only when a missed cold night is expensive relative to acting. At $L/C = 10$ the 10% interval rule saves money against doing nothing in 99.8% of resamples and beats the LSTM point rule in 94%, but against the persistence habit it is positive in only 94% and its 5th percentile is slightly negative. At $L/C = 20$ it beats the habit in every resample. At $L/C = 5$ it is worse than the habit in 98% of resamples. So the interval does not simply make decisions "more reliable"; it lets the operator trade false alarms for missed events, and the right trade depends on L/C.

How wrong can it be, and what would it cost? At $L/C = 10$ the rule acts on 233 of 548 nights and 187 of those actions were unnecessary (cost $187C$), while it missed 3 cold nights (cost $3L = 30C$). If the real L/C is about half of what we assume (around 5), the same rule costs more than the habit. Choosing the rule on validation instead of test cost at most about 10 to 11% more than the best rule. The LSTM point rule is the robust but small win: it saves in at least 99.8% of resamples against doing nothing at all three ratios, but most of that saving is available for free from the persistence habit, which saves 21%, 31% and 36% against doing nothing at $L/C = 5$, 10 and 20.

Limitations. One test period; 49 cold nights that cluster; a threshold (6°C) chosen after seeing the counts; L/C assumed; one random seed per neural configuration; a small random search; and a lag range (maximum 30 days) that may be too short for the networks to infer the season.

8 Management assessment

How effective is the solution? For a site where a missed cold night costs more than about ten times a night of acting, the interval rule roughly halves the cost of the decision: 46% less than doing nothing and 22% less than the simple habit at $L/C = 10$ (62% and 43% at $L/C = 15$). The saving grows with L/C and is limited by false alarms, since the rule acts on 43% of nights.

What is it worth? With illustrative values $C = 20$ and $L = 200$ currency units, the 548-night test period would cost 9,800 with no action, 6,780 with the habit and 5,260 with the interval rule, that is 4,540 saved against doing nothing and 1,520 against the habit, or about 3,000 and 1,000 per year. These figures are assumptions. A site that can measure its own C and L can read its saving from Table 12 or from Figure 5.

When not to use it. If L/C is below about 7, the persistence habit performs as well as the model at no cost, so we would not deploy the model. If L/C is below about 2.5, even the habit does not pay and the operator should not act on forecasts.

9 Conclusion

The best classical model (ARIMA with yearly Fourier terms) and a tuned LSTM [5] both beat persistence by 14 to 15% in RMSE and are close to each other, so non-linear modelling gives little advantage for one-step-ahead daily temperature. The more important result is about decisions: the model with the lowest RMSE was not the best tool for the cold-night decision, because point forecasts under-predict extremes. A lower-quantile forecast from the pinball-loss LSTM caught 46 of 49 cold nights at the 10% level and saved 46% of cost at $L/C = 10$, but only when missed nights are expensive. Future work would use more years of data, a longer input window or an explicit seasonal input for the networks, a measured cost for a real site, and a probability level tuned for the actual L/C.

References

[1] Box, G. E. P., Jenkins, G. M., Reinsel, G. C. and Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley.

[2] Hyndman, R. J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*, 3rd ed. OTexts.

[3] Dickey, D. A. and Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366), 427–431.

[4] Kwiatkowski, D., Phillips, P. C. B., Schmidt, P. and Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. *Journal of Econometrics*, 54, 159–178.

[5] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[6] Koenker, R. and Bassett, G. (1978). Regression quantiles. *Econometrica*, 46(1), 33–50.

[7] Kingma, D. P. and Ba, J. (2015). Adam: a method for stochastic optimization. *International Conference on Learning Representations*.

[8] Künsch, H. R. (1989). The jackknife and the bootstrap for general stationary observations. *Annals of Statistics*, 17(3), 1217–1241.

[9] Daily minimum temperatures in Melbourne, Australia, 1981–1990 (Australian Bureau of Meteorology), as distributed in the Time Series Data Library. [Add the exact source and access date you used.]

[10] Seabold, S. and Perktold, J. (2010). statsmodels: econometric and statistical modeling with Python. *Proceedings of the 9th Python in Science Conference*.

AI disclaimer

We used Claude (Anthropic), Google AI mode and ChatGPT as toolS during this project for: suggesting the project structure and the order of analysis, drafting and debugging Python code, suggesting methods (one-step-ahead evaluation with frozen parameters, Fourier terms for yearly seasonality, the block bootstrap, and the C/L quantile rule for decisions), and checking the numbers in this report against the notebook output. We ran all the code ourselves in Google Colab and the numbers in this report come from our own runs. We chose the decision context and the cost scenarios, and the observations and conclusions are based on our results.

Appendix: reproducibility

Repository: https://github.com/ikramkakar/time-series-forecasting-melbourne. The notebook runs top to bottom in Google Colab with pandas, NumPy, statsmodels [10], scikit-learn, TensorFlow/Keras and matplotlib. Fixed seeds: neural networks 42, random search 0, bootstrap 1 (2,000 resamples, 14-day blocks). Files: the notebook, the data file, `test_results.csv`, `test_predictions.csv`, `decision_table.csv`, `model_summary.csv` and the figures.